Chiron

Effect Foundation and Runtime Migration Execution on a Workflow-Path Backbone

Progress Report 8 • 5 Jan 2026 — 18 Jan 2026

Master’s Research Progress Update

Fahad Alothman

Progress Report 8

Title: Effect Foundation and Runtime Migration Execution on a Workflow-Path Backbone
Reporting Window: 5 Jan 2026 — 18 Jan 2026
Submission Context: Master's Research Progress Update

Abstract

Progress Report 8 documents the main migration-execution window that followed the strategic reframing in Progress Report 7. The period advanced a staged runtime transition from the prior orchestration center toward an Effect- and AI-SDK-oriented execution core. This was not a planning-only interval; it was an implementation-heavy phase with tightly sequenced delivery slices.

A key historical point is architectural continuity during change. Chiron was still operating with independent workflows, and workflow paths remained the practical backbone for execution flow. The migration therefore focused on replacing runtime responsibilities while preserving path-driven execution continuity and user-facing workflow operability.

---

1. Period Character: Migration Execution Under Controlled Scope

This reporting window was a high-intensity implementation period centered on runtime replacement, not broad capability expansion. The scope was controlled through story-sliced migration increments, each targeting one dependency layer or execution concern.

The practical objective was to reduce orchestration indirection while improving ownership over lifecycle, variable flow, approval semantics, and service boundaries. The period should be interpreted as controlled architectural execution rather than exploratory prototyping.

---

2. Execution Model Continuity: Independent Workflows with Workflow Paths as Backbone

At this stage, Chiron's working execution model remained anchored in independent workflows connected by workflow paths; formal methodology-layer contracts had not yet become the primary execution model. In this period, that backbone was retained deliberately.

This decision mattered for delivery stability:

- execution continuity remained path-oriented,
- migration work could proceed without rebuilding every user-facing orchestration surface at once,
- and runtime internals could be replaced while preserving operational behavior at the workflow-navigation level.

In other words, the project changed the engine while keeping the route structure understandable and usable.

---

3. Effect Foundation: Establishing Service-Layer Ownership

The first major outcome of the period was the establishment of an Effect-based foundation for the workflow engine [1]. Core execution responsibilities were reorganized into explicit services and layered runtime composition.

This strengthened determinism and maintainability by moving critical concerns into typed, composable service boundaries instead of dispersed orchestration logic. The architecture gained clearer control points for configuration, execution context, error handling, and stateful runtime interaction.

---

4. AI-SDK Integration and Orchestration Decoupling

The second major outcome was integration of AI-SDK-centered interaction primitives into the new runtime path [2], [3]. This was accompanied by active removal of prior orchestration coupling and expansion of multi-provider AI handling.

Operationally, this reduced dependence on a heavier abstraction path and improved direct control over message flow, tool execution interfaces, and provider wiring. The period therefore converted earlier strategic intent into concrete runtime behavior.

---

5. Variable, Step-Handler, and Executor Migration Sequence

A central implementation sequence in this window addressed the core execution loop in stages:

- variable-system migration into the new runtime layer,
- step-handler migration into Effect-aligned service execution,

- mainline wiring of the new executor path,
- and cleanup of legacy execution surfaces after replacement stabilized.

This sequence reduced migration risk by avoiding a single-shot runtime cutover. It also improved traceability of behavior changes because each layer of the execution loop was transitioned with explicit scope.

---

6. Scoping Strategy in This Period: Operational Boundaries Before Formal Methodology Boundaries

The project's scoping strategy in this report window was pragmatic: enforce control through runtime services and execution-path constraints while formal methodology-layer contracts were still emerging.

This means scope was managed through operational authority boundaries:

- where variable ownership lived,
- where transition and approval decisions executed,
- and where workflow-step context was interpreted and persisted.

The significance is historical: the period did not finalize the later formal methodology-layer model, but it made the boundaries that model would later codify visible and testable in implementation practice.

---

7. Stack Status at End of Progress Report 8

By the end of this reporting window, the active runtime direction had shifted noticeably in implementation practice:

- **Desktop host:** Tauri [5]
- **Frontend:** React + TanStack Router + shadcn/ui
- **Backend/API:** TypeScript services with Hono + tRPC [6], [7]
- **AI/runtime orchestration path:** Effect + AI-SDK (+ AX integrations where needed) [1]–[4]
- **Data layer:** Drizzle ORM with PostgreSQL
- **Authentication:** Better Auth
- **Runtime/tooling:** Bun + Turborepo

Compared with the prior report, this period marks execution-level migration progress rather than only directional reassessment.

8. Why This Period Matters

Progress Report 8 matters because it operationalized the transition logic established earlier. It showed that runtime migration could proceed while preserving workflow-path continuity and without immediate dependence on a fully materialized methodology layer.

This period also clarified the next requirement: once runtime boundaries were made explicit in services and execution flow, they needed to be elevated into stable methodology-level contracts. That is the bridge to subsequent lifecycle/state formalization work.

---

9. Open Questions Entering Progress Report 9

- Which operational scoping rules from runtime services should be promoted into first-class methodology-layer contracts?
- How should project-, workflow-, and step-level authority be represented so path-driven execution remains coherent as lifecycle complexity grows?
- Which transition and approval semantics are now stable enough to standardize across workflows?
- What verification strategy is required to preserve migration gains while moving toward formalized lifecycle/state models?

These questions define the handoff from migration execution to boundary formalization.

Conclusion

Progress Report 8 is the implementation counterpart to the reframing captured in Progress Report 7. The period replaced core runtime surfaces through a staged Effect and AI-SDK migration while preserving independent-workflow operation through workflow paths.

Its central contribution is not only technology substitution, but boundary exposure: it made scoping, transition, and authority concerns concrete in running systems, preparing the ground for the formal methodology-layer and lifecycle/state model that followed.

References

[1] Effect Authors, “Effect Documentation,” 2026. [Online]. Available: https://effect.website/
[2] Vercel, “AI SDK Documentation,” 2026. [Online]. Available: https://ai-sdk.dev/docs
[3] Vercel, “AI SDK 6,” 2025. [Online]. Available: https://vercel.com/blog/ai-sdk-6
[4] axllm, “ax,” GitHub repository (`@ax-llm/ax`), 2026. [Online]. Available: https://github.com/axllm/ax
[5] Tauri Programme, “Tauri,” Documentation, 2026. [Online]. Available: https://tauri.app/
[6] Hono Authors, “Hono,” Documentation, 2026. [Online]. Available: https://hono.dev/
[7] tRPC Authors, “tRPC,” Documentation, 2026. [Online]. Available: https://trpc.io/docs